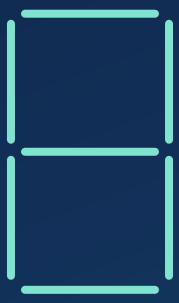
DIGITAL LOGIC DESIGN • PROJECT REPORT



4-Bit Binary to Seven-Segment Display Decoder Using Logic Gates

A Sum-of-Products (SOP) Combinational Logic Implementation, Designed & Simulated in Proteus Professional

Prepared by: Zeerak Butt | **Programme:** BS Computer Engineering

Institution: Bahria University, H-11 Campus, Islamabad

Course: Digital Logic Design | **Simulation Tool:** Proteus Professional 8.x

GITHUB • 4-bit-binary-to-7-segment-decoder

Abstract

This project presents the design and implementation of a **4-Bit Binary to Seven-Segment Display Decoder** using digital logic gates in Proteus Professional. Instead of relying on a dedicated decoder IC, the circuit is built entirely from Boolean algebra and Sum-of-Products (SOP) expressions obtained through Karnaugh Map (K-Map) minimization. The system accepts a 4-bit binary input (0000–1111) and drives a common-cathode seven-segment display to show the corresponding hexadecimal digit (0–F). The project demonstrates practical combinational logic design, Boolean simplification, and digital circuit simulation from first principles.

Combinational Logic | K-Map Minimization | SOP Design | No Decoder IC | Proteus Simulation

Introduction

A seven-segment display is one of the most widely used output devices in digital electronics for showing decimal and hexadecimal digits. It consists of seven individually controllable LED segments, labelled **a** through **g**, arranged to form numerals when specific combinations are illuminated.

Conventionally, dedicated decoder ICs such as the 7447 or 4511 are used to convert a binary input into the correct seven-segment pattern. In this project, however, the decoding logic is implemented manually using elementary logic gates (AND, OR, NOT). The Boolean expression for each segment is derived independently using Karnaugh Maps, minimized into its simplest Sum-of-Products form, and then realized as a gate-level circuit — allowing the full binary-to-hex decoding function to work without any decoder IC. The complete circuit is designed, wired, and simulated in Proteus Professional.

Objectives

- Design a 4-bit binary input stage using logic state switches.
- Decode and display hexadecimal values (0–F) on a seven-segment display.
- Implement the decoder logic using only discrete logic gates — no decoder IC.
- Apply Boolean algebra and Karnaugh Map minimization to derive optimal SOP expressions.
- Simulate, verify, and validate the complete design in Proteus Professional.

Components Used

Component	Quantity	Purpose
Logic State Switches	4	Provide 4-bit binary input (A, B, C, D)
AND Gates	Multiple	Implement product (minterm) terms
OR Gates	Multiple	Sum the product terms for each segment
NOT Gates (Inverters)	Multiple	Generate complemented input literals
Seven-Segment Display	1	Common-cathode hexadecimal output display
Power Supply (5V)	1	Circuit voltage source
Ground Reference	1	Common return path
Proteus Professional	Software	Schematic capture & simulation environment

System Block Diagram

Working Principle

The circuit consists of four manual logic state switches that provide the 4-bit binary input, labelled **A** (MSB), **B**, **C**, and **D** (LSB). Toggling these switches between logic 0 and logic 1 produces every combination from 0000 to 1111, corresponding to hexadecimal digits 0 through F.

Each binary input is fed — in both true and complemented form — into a network of AND and OR gates. Every segment of the display (a, b, c, d, e, f, g) is driven by its own independent Boolean SOP expression, obtained by minimizing the corresponding K-Map. Whenever the switch combination changes, the gate network re-evaluates all seven expressions simultaneously and drives the correct segments, causing the display to update instantly.

OBSERVATION — SWITCH / SEGMENT BEHAVIOUR

In this build, the segment outputs respond according to the logic level applied at the switches: driving a switch to **logic HIGH** forces its associated gate branch high, which in turn switches the connected segment(s) into their **illuminated (active)** state, while a **logic LOW** input holds the corresponding branch — and therefore its segment output — in the **OFF/low** state. Since every segment is controlled by a shared combination of input literals, changing even one switch can affect several segments at once, so it is important to verify each transition against the truth table below rather than assuming a single switch maps to a single segment.

Truth Table (Binary Input → Segment Outputs)

1 = segment ON (illuminated), 0 = segment OFF | A = MSB, D = LSB

A	B	C	D	Hex	a	b	c	d	e	f	g
0	0	0	0	0	1	1	1	1	1	1	0
0	0	0	1	1	0	1	1	0	0	0	0
0	0	1	0	2	1	1	0	1	1	0	1
0	0	1	1	3	1	1	1	1	0	0	1
0	1	0	0	4	0	1	1	0	0	1	1
0	1	0	1	5	1	0	1	1	0	1	1
0	1	1	0	6	1	0	1	1	1	1	1
0	1	1	1	7	1	1	1	0	0	0	0
1	0	0	0	8	1	1	1	1	1	1	1
1	0	0	1	9	1	1	1	1	0	1	1
1	0	1	0	A	1	1	1	0	1	1	1
1	0	1	1	B	0	0	1	1	1	1	1
1	1	0	0	C	1	0	0	1	1	1	0
1	1	0	1	D	0	1	1	1	1	0	1

1	1	1	0	**E**	1	0	0	1	1	1	1
1	1	1	1	**F**	1	0	0	0	1	1	1

Karnaugh Map – Derived Boolean Expressions

Each of the seven segment outputs was individually minimized from its 16-cell Karnaugh map (variables A, B, C, D) into a reduced Sum-of-Products expression. These are the exact equations implemented by the gate network in the Proteus schematic:

Segment A (Top) $A = BC + A\overline{D} + C\overline{A} + \overline{BD} + BD\overline{A} + A\overline{BC}$

Segment B (Top-Right) $B = \overline{AB} + \overline{BD} + AD\overline{C} + CD\overline{A} + \overline{ACD}$

Segment C (Bottom-Right) $C = A\overline{B} + B\overline{A} + D\overline{A} + D\overline{C} + \overline{AC}$

Segment D (Bottom) $D = A\overline{C} + BC\overline{D} + BD\overline{C} + CD\overline{B} + \overline{ABD}$

Segment E (Bottom-Left) $E = AB + AC + C\overline{D} + \overline{BD}$

Segment F (Top-Left) $F = AC + A\overline{B} + B\overline{D} + \overline{CD} + B\overline{AC}$

Segment G (Middle) $G = AD + A\overline{B} + C\overline{B} + C\overline{D} + B\overline{AC}$

Notation: a bar over a variable (e.g. $\overline{A}$) denotes its logical complement (NOT A); adjacent letters denote an AND operation; " + " denotes OR.

Circuit Diagram – Proteus Schematic

Fig. 1 — Complete gate-level SOP implementation: 4-bit input switches (top-left) feed a network of AND/OR gates that drive the seven-segment display (top-right), simulated in Proteus Professional.

Features

- No decoder IC used — pure gate-level design
- Full hexadecimal range (0–F) supported
- Minimized SOP implementation via K-Maps
- Manual switch-based input for live testing
- Fully simulated and verified in Proteus
- Modular, segment-independent logic design

Advantages

- Provides an intuitive, hands-on understanding of combinational logic design.
- Demonstrates practical application of Boolean algebra and K-Map minimization.
- Reduces dependence on dedicated decoder ICs such as the 7447/4511.
- Highly suitable for Digital Logic Design laboratory coursework and demonstrations.

Applications

- Digital Electronics teaching laboratories
- Educational demonstrations of Boolean logic
- FPGA logic verification & prototyping
- Embedded systems learning modules

Software Used

Proteus Professional 8.x was used for schematic capture, gate-level circuit construction, and real-time simulation of the decoder against all 16 possible input combinations.

Future Improvements

- Re-implement the entire design using NAND gates only (universal gate realization).
- Port the logic design into Verilog HDL for synthesis-based verification.
- Target an FPGA platform for hardware-level deployment.
- Add a dedicated hexadecimal enable/latch switch for controlled display updates.
- Progress the verified design into a physical PCB implementation.

Conclusion

This project successfully implements a 4-Bit Binary to Seven-Segment Display Decoder using only elementary logic gates. Boolean expressions obtained through Karnaugh Map minimization were realized as an AND-OR gate network to independently control each segment of the display. Simulation in Proteus Professional confirms that the circuit correctly displays the appropriate hexadecimal digit for every one of the sixteen possible 4-bit input combinations. The project reinforces core concepts in combinational logic design, Boolean simplification, and practical digital circuit implementation — skills directly applicable to further coursework in embedded systems and digital hardware design.